

Week 01: MDPs, Rollouts, and Baselines

The Foundations of Controlled Interaction Under Uncertainty

Reinforcement Learning & Robot Learning Core Curriculum

Presented by **DGIST ISL Team**

DGIST Intelligent Systems and Learning Laboratory

2026.08.21

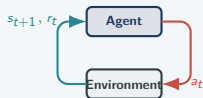
Core Concepts: MDP Tuple · Agent-Environment Loop · Episode Return · Random Baseline

The Agent-Environment Interface & MDP Tuple

Reinforcement learning is structured sequential decision-making

The Interaction Loop

- **Agent:** Observes state s_t , selects action $a_t \sim \pi(\cdot | s_t)$.
- **Environment:** Emits next state s_{t+1} , reward r_t , and ending signals.
- Agent controls *actions*; environment governs *dynamics*.

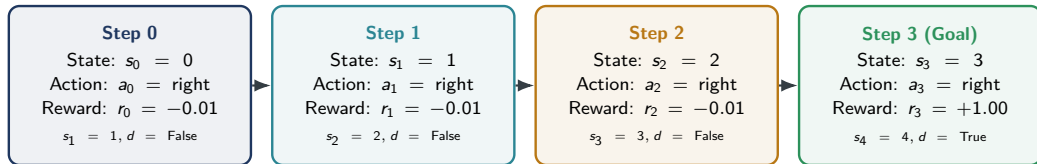


The Formal MDP Tuple: $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$

- $\mathcal{S}$: State space (discrete positions or continuous poses).
- $\mathcal{A}$: Action space (discrete commands or motor torques).
- $\mathcal{P}(s' | s, a)$: Transition probability function.
- $\mathcal{R}(s, a, s')$: Reward function mapping steps to scalar feedback.
- $\gamma \in [0, 1]$: Discount factor prioritizing near-term rewards.

Anatomy of an Episode Rollout & Trajectory

How step-level transitions aggregate into total episode return



Undiscounted Return: $G = \sum_{t=0}^T r_t$

$$G = (-0.01) + (-0.01) + (-0.01) + (+1.00) = +0.97.$$

Negative step costs penalize hesitation.

Discounted Return: $G = \sum_{t=0}^T \gamma^t r_t$

For $\gamma = 0.90$:

$$G = -0.01 - 0.01(0.9) - 0.01(0.81) + 1.00(0.729) = +0.7019.$$

The LineWorld Benchmark Environment

A minimal 1D discrete MDP for rigorous baseline comparison



MDP Rules

- $\mathcal{S} = \{0, 1, 2, 3, 4\}$, start state $s_0 = 0$.
- $\mathcal{A} = \{0 : \text{left}, 1 : \text{right}\}$.
- Boundaries: moving left at $s = 0$ stays at $s = 0$.
- Step cost: $r = -0.01$ for every non-goal transition.

Termination Signals

- **Terminated** (d_{term}): Reaching goal state 4 ($r = +1.00$).
- **Truncated** (d_{trunc}): Exceeding maximum horizon $T_{\text{max}} = 20$.
- Clear distinction between task success and horizon timeout.

The Role of Baselines in Reinforcement Learning

Why random and heuristic baselines are required before claiming learning success

Policy Type	Discovery Mechanism	Avg Return	Success Rate	Purpose
Random Policy	Uniform random actions	+0.52	65%	Lower bound (chance)
Always Right (Heuristic)	Hard-coded domain prior	+0.97	100%	Known-task reference
Learned Policy (Weeks 2–4)	Discovered from rewards	+0.97	100%	Autonomous discovery

Random Baseline Discipline

A learning algorithm that achieves 60% success on LineWorld has learned **nothing**—random actions succeed 65% of the time!

Heuristic Comparison

Heuristics show what is possible with optimal domain knowledge, providing an empirical benchmark for learning methods.

Environment Protocol: Terminated vs Truncated

Standard Gymnasium API conventions for episode boundaries

Terminated ($d_{\text{term}} = \text{True}$)

- Natural endpoint of the MDP (goal reached or robot fell).
- Future expected return from this state is **strictly zero**.
- In value updates: Target $y = r$ (no bootstrapping).
- Represents true task completion.

Truncated ($d_{\text{trunc}} = \text{True}$)

- Artificial cutoff due to time limit ($t \geq T_{\text{max}}$).
- State is **not terminal**; agent could continue.
- In value updates: Target $y = r + \gamma \max Q(s', \cdot)$ (must bootstrap!).
- Treating an external time limit as task termination can bias value targets by discarding continuation value.

Code Lens: Inspecting a Tiny MDP in Python

The canonical environment interaction pattern in examples/week-01

Canonical Environment Loop

Python Execution Pattern

```
env = LineWorld(size=5, max_steps=20)
state = env.reset(start=0)
done = False; ep_return = 0.0
while not done:
    action = policy(state)
    res = env.step(action)
    ep_return += res.reward
    state = res.state
    done = res.terminated or res.truncated
```

Key Takeaways

- Three core steps: **observe** → **act** → **step**.
- Track scalar return $G = \sum r_t$.
- Cleanly separate agent decision logic from environment transition logic.
- Transparent, zero external dependencies.

Failure Modes in MDP Formulation

Common pitfalls when formulating tasks and evaluating baselines

1. SILENT INFINITE LOOPS

Without max step limits ($T_{\max}$), random exploration can wander indefinitely without terminating or raising errors.

2. REWARD HACKING / CHEATING

Positive rewards for intermediate states encourage looping behaviors rather than reaching the true goal efficiently.

3. RETURN VS SUCCESS CONFUSION

Reporting only success rate hides efficiency; a policy that takes 19 steps has lower return than one taking 4 steps.

4. MISSING RANDOM BASELINES

Evaluating complex algorithms without measuring random performance leads to false claims of algorithmic capability.

Robotics Bridge: Formulating Real Systems as MDPs

Mapping continuous physical robotic systems into the formal MDP framework

Physical Robot MDP Mapping

- **State $\mathcal{S}$:** Joint positions $\theta \in \mathbb{R}^7$, velocities $\dot{\theta}$, end-effector pose $T_{ee} \in \text{SE}(3)$, tactile forces.
- **Action $\mathcal{A}$:** Motor torques $\tau \in \mathbb{R}^7$ or delta Cartesian setpoints $\Delta x \in \mathbb{R}^6$.
- **Transitions $\mathcal{P}$:** Governed by rigid body physics, contact dynamics, and motor inertia.

Reward Engineering in Robotics

- **Dense Rewards:** Distance to target $-\|x_{ee} - x_{\text{target}}\|_2$ (smooth gradients, risk of unintended exploits).
- **Sparse Rewards:** +1 on successful grasp (unbiased, but difficult exploration).
- **Safety Costs:** Penalty for exceeding joint torque limits or collisions.

Week 01 Quiz: Core Concepts & Interaction Loop

Self-check: verify fundamental understanding of MDPs and baselines

Q1. Role of the Random Baseline

Why is evaluating a uniform random policy mandatory before testing learned agents?

- A. To initialize neural network weights.
- B. To establish a baseline of chance performance before adding heuristics or learning.
- C. To ensure exploration covers all possible states.

Answer: B $\implies$ Empirical baseline discipline.

Q2. Reward vs. Return

What is the precise mathematical difference between reward and return?

- A. Reward is for continuous tasks; return is for discrete tasks.
- B. Reward is immediate feedback from one step (r_t); return is the accumulated sum of rewards over an episode ($G = \sum \gamma^t r_t$).
- C. Return is always positive; reward can be negative.

Answer: B $\implies$ Step feedback vs objective.

Week 01 Quiz: Rollout Reading & Return Arithmetic

Verify hand-computations for trajectory returns and step costs

Problem 1: Undiscounted Episode Return

Trajectory: $s_0 = 0 \xrightarrow{R} s_1 = 1 \xrightarrow{R} s_2 = 2 \xrightarrow{R} s_3 = 3 \xrightarrow{R} s_4 = 4(\text{Goal})$

Step Rewards: $r_0 = -0.01, r_1 = -0.01, r_2 = -0.01, r_3 = +1.00$

Calculate: Total undiscounted return G .

Solution:

- $G = r_0 + r_1 + r_2 + r_3$
- $G = -0.01 - 0.01 - 0.01 + 1.00 = +\mathbf{0.97}$
- Goal reached in minimal 4 steps.

Problem 2: Timeout Truncation Return

Scenario: Agent takes 20 random steps without reaching the goal ($T_{\max} = 20$).

Step Rewards: $r_t = -0.01$ for all 20 steps.

Calculate: Episode return and termination status.

Solution:

- $G = 20 \times (-0.01) = -\mathbf{0.20}$
- Status: $d_{\text{term}} = \text{False}, d_{\text{trunc}} = \text{True}$
- Reflects horizon timeout without goal reward.

Week 01 Quiz: Diagnostics & Practical Traps

Key practical questions every RL practitioner must master

Q3. Why Isn't "Always Right" a Learning Method?

It hardcodes human prior knowledge about goal location rather than discovering optimal actions from experience.

Q4. Why Track Return Over Success Rate?

Success rate is binary; return measures path efficiency by penalizing wasted steps via negative step costs.

Q5. Effect of Setting Non-Goal Reward to 0

Without negative step costs ($r = 0$ instead of -0.01), the agent has no incentive to reach the goal quickly.

Q6. Terminated vs Truncated in Learning

Task termination drops the bootstrap term; an external truncation ends the rollout but usually retains continuation value.